

Predictive Action Projection with PPO in Constrained Continuous Control

Implementation Design and Experimental Verification Contract

Salvador Tenorio

Implementation specification

Abstract

This document is the implementation-facing technical design for a compact simulated study of predictive action projection with Proximal Policy Optimization (PPO). It specifies the mathematical model, software architecture, action ownership, control-barrier-function quadratic program, PPO rollout semantics, training variants, evaluation workflows, artifact schemas, aggregation hierarchy, validation architecture, and change-control boundaries implemented for the study. The system separates the unbounded normalized action sampled and scored by PPO, the bounded normalized action produced by componentwise clipping, the physical nominal action produced by the action wrapper, and the physical action executed after optional CBF-QP projection. This four-stage action contract is central to both algorithmic correctness and interpretation of filter dependence. The document defines what the implementation does and what its verification workflows must establish; it does not report whether a particular source revision has passed validation, summarize experiment-campaign progress, or present empirical conclusions. It contains no design for subsequent structured-model, Gauss–Newton, or offline-RL projects.

Contents

1	Purpose, authority, and document boundaries	4
1.1	Design role	4
1.2	Documentation hierarchy	4
1.3	Supersession boundary	4
1.4	Status and evidence boundary	5
1.5	Scope	5
2	Scientific objective and design rationale	5
2.1	Study objective	5
2.2	Implemented comparison structure	5
2.3	Principal research questions	6
2.4	Major design decisions	6
2.5	Claim boundaries	7
3	System architecture and ownership	8
3.1	Repository-level workflow	8
3.2	Ownership invariants	8
3.3	Primary module map	9
3.4	Dependency direction	11

4	Environment and task contract	11
4.1	State and fixed-capacity observation	11
4.2	Physical actions and dynamics	11
4.3	Collision, success, and episode flags	11
4.4	Reward	12
4.5	Reset and controlled layouts	12
4.6	Environment-level invariants	13
5	Action interfaces and wrapper topology	13
5.1	Wrapper nesting	13
5.2	Four-stage action semantics	14
5.3	Normalized action-bound diagnostics	15
5.4	Action-path invariants	15
6	Predictive action projection	15
6.1	Lookahead geometry	15
6.2	Collision and projection radii	16
6.3	Affine CBF rows	16
6.4	Projection QP	16
6.5	Projection result contract	16
6.6	Special paths and failure semantics	17
6.7	Interpretive meaning of diagnostics	18
7	PPO implementation and safeguarded-environment semantics	18
7.1	Actor-critic architecture	18
7.2	Effective projected environment	19
7.3	Vector rollout and same-step autoreset	19
7.4	GAE and PPO-Clip	19
7.5	Training diagnostics	21
7.6	PPO selection relative to SAC	21
7.7	Checkpoint contract	21
8	Training variants, configuration, and campaign controls	21
8.1	Shared launcher	21
8.2	Training methods	22
8.3	Protocol-controlled evaluation matrix	22
8.4	Development protocol	23
8.5	Training device benchmark	23
8.6	Independent replication unit	23
9	Evaluation architecture	24
9.1	Evaluation modes	24
9.2	Episode result schema	24
9.3	Single-policy evaluation algorithm	25
9.4	Same-checkpoint paired evaluation	25
9.5	Named-layout evaluation	25
9.6	Metric interpretation	26
10	Layout suites, calibration, and benchmark identity	26
10.1	Layout suite schema	26
10.2	Semantic hashing	26
10.3	Development suite	27
10.4	Frozen core suite	27
10.5	Calibration procedure	27
10.6	Calibration record boundary	28
10.7	Evaluation-level separation	28

11 Artifact contracts and provenance	28
11.1 Artifact lifecycle	28
11.2 Checkpoint identity	29
11.3 TensorBoard event contract	29
11.4 Episode CSV contract	29
11.5 Trajectory archive contract	29
11.6 Analysis protocol contract	30
11.7 Generated output contract	30
12 Analysis and statistical aggregation	30
12.1 Metadata-driven discovery	30
12.2 Checkpoint-first hierarchy	31
12.3 Paired projection deltas	31
12.4 Training diagnostic exports	31
12.5 Scientific diagnostic figures	31
13 Orchestration and validation architecture	32
13.1 Python orchestration and Windows launchers	32
13.2 Validation layers	32
13.3 Completed-release verification	33
13.4 Lightweight test allocation	33
13.5 PASS-marker semantics	34
13.6 Verification responsibility	34
13.7 Evidence separation	34
14 Traceability from mathematics to evidence	35
14.1 Contract traceability matrix	35
14.2 Change-impact rules	37
15 Default parameters and rationale	37
16 Failure semantics and research incidents	38
16.1 Explicit failure table	38
16.2 Incident handling principle	39
17 Limitations and interpretation boundaries	39
18 Change control and experiment freeze	40
18.1 Validity-threatening changes	40
18.2 Contracts requiring explicit revision	40
18.3 Freeze checklist	40
A Canonical command entry points	41

1 Purpose, authority, and document boundaries

1.1 Design role

This report is the authoritative consolidated design for the implemented predictive action-projection study. Its purpose is to make the repository understandable and auditable at the level required to conduct, reproduce, and later defend the experiments. It records stable mathematical definitions, component responsibilities, data identities, failure semantics, algorithms, and evidence flows. It is not a programming tutorial, a chronological development diary, or a presentation of experimental results.

The codebase remains the final source of truth for runtime behavior. A disagreement between code and this document is treated as a documentation or implementation defect to be resolved explicitly; prose is not used to rationalize obsolete behavior. Executable tests and validators determine whether a particular source revision satisfies the declared contracts.

1.2 Documentation hierarchy

The repository documentation has distinct roles:

Artifact	Primary role	Authority boundary
Implementation design	Consolidated mathematical, architectural, algorithmic, and verification specification.	Authoritative design when synchronized with the implementation.
Software and artifact companion	Operational mental models, code-reading guidance, and artifact interpretation.	Pedagogical; subordinate to this design, focused contracts, protocols, validators, and code.
Markdown contracts	Exact interface, schema, metric, and failure semantics for focused subsystems.	Must agree with code and this design; changes require review.
Validation guide	Supported executable gates, output paths, and PASS-marker semantics.	Validators and tests are executable sources of truth for a selected source revision.
Research records	Regression, calibration, freeze, and provenance decisions tied to named artifacts.	Immutable evidence; not part of this design document's status.
Analysis protocols	Machine-readable declaration of methods, seeds, modes, suites, and required evidence.	Authoritative for a particular experiment and result build.
Separate scientific paper	Scientific context, experiment report, results, and conclusions.	Empirical claims depend on frozen audited artifacts.

1.3 Supersession boundary

This document supersedes earlier implementation specifications for the present study, including obsolete paths, milestone labels, fallback behavior, and action semantics. Earlier research-program reports remain historical records of motivation and future-project planning, but they are not authoritative descriptions of the current repository. In particular, this document does not restore the former multi-project scope.

1.4 Status and evidence boundary

This document describes implemented behavior and required verification workflows. It is not a validation report, a protocol-freeze record, an experiment log, or a summary of results. A validation status belongs to generated evidence tied to a specific source revision and environment; a campaign definition belongs to a machine-readable protocol and its freeze record; empirical outcomes belong to audited result artifacts and the scientific report. Descriptions of tests, validators, calibration, and freeze procedures in this document are therefore normative descriptions of what those workflows must do, not claims that a particular run has completed.

1.5 Scope

The repository implements a compact Python simulation for constrained continuous control. Its active scope includes:

- a fixed-shape Gymnasium navigation environment with unicycle dynamics;
- a CleanRL-style PPO actor-critic implementation;
- a normalized action interface with explicit action-bound clipping diagnostics;
- an optional softened CBF-QP projection layer in physical action coordinates;
- baseline, high-collision-penalty, and projection-enabled training variants;
- deterministic and stochastic checkpoint evaluation;
- same-checkpoint projection-off/on pairing;
- validated named layout suites and trajectory recording;
- calibration, validation, device benchmarking, aggregation, and scientific diagnostic plots.

The active implementation does not include rendering, workspace boundaries, hardware, ROS, real-time control, a global planner, a differentiable QP layer, a corrected projected PPO estimator, or any later research project.

2 Scientific objective and design rationale

2.1 Study objective

The study examines the interaction among a learned continuous-control policy, local action correction, task performance, and reliance on the correcting filter. A policy may learn to reach a goal under reward shaping while still proposing actions that are locally hazardous. A projection layer can alter those actions before execution, but collision reduction alone does not establish that the policy itself has learned safer behavior.

The implemented evidence therefore separates three outcome families:

$$\text{task performance,} \quad \text{safety behavior,} \quad \text{intervention burden.} \quad (1)$$

Task performance includes return, success, timeout, and episode length. Safety behavior includes collision and signed clearance. Intervention burden includes normalized action-bound clipping, physical projection intervention, correction magnitude, slack, and solver status.

2.2 Implemented comparison structure

The implemented training methods are:

1. `ppo_baseline`: PPO with the default collision penalty and no training-time projection;
2. `ppo_high_penalty`: PPO with a larger collision penalty and no training-time projection;

3. `ppo_train_projection`: PPO interacting with the physical CBF-QP projection layer during training.

Evaluation-time projection is an independent deployment choice. Both completed-study protocols require Projector OFF and Projector ON for all three training histories. Five independently initialized runs per history, each represented by one retained final checkpoint, provide all six history-by-runtime cells. The machine-readable protocols remain authoritative; this document describes their frozen configuration without changing it.

The OFF/ON evaluation conditions and transfer layouts were fixed before the final outcomes were observed. The paper-level decision to organize the completed evidence around learned-policy versus composite-controller attribution was retrospective. The paper and its production sources are maintained separately from this code repository.

2.3 Principal research questions

The implemented system is capable of addressing the following questions without implying their answers:

1. How does evaluation-time projection change collision, success, timeout, clearance, and return for the same frozen checkpoint?
2. Does training under projected transitions alter task behavior or reduce dependence on the filter?
3. How does a larger collision penalty compare with direct action correction under a common evaluation reward?
4. Does the policy distribution frequently exceed its normalized action bounds before the CBF-QP is reached?
5. Does a learned policy transfer from the fixed training geometry to a frozen suite of changed obstacle layouts?
6. When projection prevents collision, does it also enable route completion, or does the outcome become a timeout or another failure?

2.4 Major design decisions

Decision	Alternative or concern	Implemented rationale and consequence
PPO as the learning algorithm	SAC is often more sample efficient in continuous control.	PPO provides a compact, inspectable on-policy rollout path whose raw sampled action and likelihood can remain explicit when the environment executes a projected action. The choice prioritizes attribution and auditability rather than claiming algorithmic superiority. CleanRL reduces code size for either method, but SAC would add replay, twin critics, target networks, entropy-temperature dynamics, and critic learning over the many-to-one projection map [5, 7, 4].
Physical action projection	Collision penalty alone.	A penalty influences learning but does not prevent a proposed command from being executed. Projection creates a distinct intervention mechanism and measurable correction burden.

Decision	Alternative or concern	Implemented rationale and consequence
Lookahead-point barrier	Agent-center velocity only.	A lookahead point makes both forward speed and turn rate enter the local obstacle constraint, which is useful for a unicycle model.
Soft CBF constraints	Hard constraints.	Nonnegative slack avoids treating local geometric conflict as a silent solver crash and makes relaxation burden observable. Softening also prevents a global safety claim.
Stationary failure action	Execute the uncorrected action after solver failure.	A failed solver result is not treated as a valid safety decision. The implementation executes $[0, 0]$, records failure, and preserves active slack as unknown. Projected training aborts.
Four-stage action model	Collapse all actions into a single “raw action.”	The actor samples an unbounded Gaussian value, the wrapper clips it, an affine map creates the physical nominal command, and the projector may alter that command. Separating these stages prevents likelihood, clipping, and projection metrics from being conflated.
Fixed training layout	Randomized task distribution.	A fixed compact task reduces experimental degrees of freedom and supports controlled learning comparisons. It limits generalization claims, so transfer is evaluated separately.
Frozen named transfer suite	Select layouts after observing final policies.	Semantic hashing, calibration before final training, and a freeze record prevent outcome-driven benchmark selection.
Checkpoint-level replication	Treat layouts or episodes as independent method replicates.	Independent training runs are the empirical units, each represented by one retained final checkpoint. Repeated observations are averaged within a run before aggregation across runs.
Saved-artifact analysis	Plot directly from live runs.	Training, evaluation, and plotting are separated. Final tables and figures are regenerated from audited immutable evidence.

2.5 Claim boundaries

The implementation can support empirical statements about measured task outcomes, collisions, clearances, action-bound saturation, projection intervention, correction burden, slack, and transfer within the declared benchmark. It does not by itself support claims of global forward invariance, optimal safe policy learning, unbiased clipped PPO under projection, general superiority over safe RL or MPC methods, broad task-distribution generalization, or real-world readiness. The projection formulation is related to CBF-QP safety filters [1] and the learning mismatch analyzed by Gros, Zanon, and Bemporad [3]; the present system deliberately maintains a conservative empirical interpretation.

3 System architecture and ownership

3.1 Repository-level workflow

The repository is an experimental numerical system rather than a single deployed application. Training creates checkpoints and event logs. Evaluation creates episode tables and trajectories. Calibration and validation are designed to create auditable evidence and focused review artifacts. Analysis consumes saved evidence and creates scientific tables and diagnostic figures.

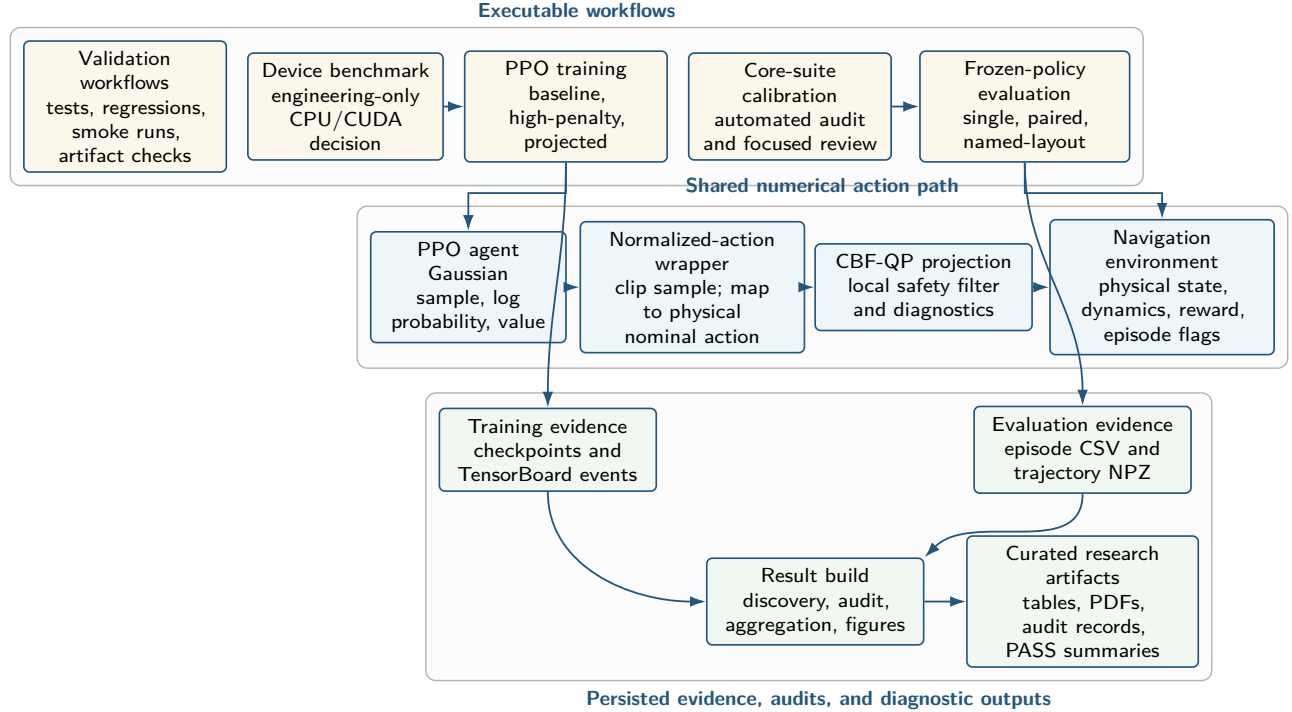


Figure 1: Component architecture and evidence flow.

3.2 Ownership invariants

The following invariants organize the implementation:

1. **Environment ownership.** `ConstrainedNavigationEnv` is the only component that advances physical state and computes task reward and terminal conditions.
2. **Policy ownership.** The PPO actor owns the unbounded normalized sample a_t^{sample} and its log probability.
3. **Bound-map ownership.** `NormalizedActionWrapper` owns componentwise clipping to a_t^{bounded} and the affine conversion to u_t^{raw} .
4. **Projection ownership.** The optional projection wrapper owns the map $u_t^{\text{raw}} \mapsto u_t^{\text{exec}}$ and the associated solver and burden diagnostics.
5. **Evaluation immutability.** Evaluation loads frozen parameters and performs no learning.
6. **Analysis immutability.** Analysis consumes saved artifacts and never launches training or policy evaluation.
7. **Orchestration reuse.** Experiment and validation modules call the same public training, evaluation, and analysis paths used by the study; they do not maintain independent algorithm copies.

3.3 Primary module map

Path	Implemented responsibility
<code>environments/constrained_navigation.py</code>	Physical-action Gymnasium environment; state, observation encoding, default and controlled layouts, explicit-Euler dynamics, reward, success, collision, truncation, and task diagnostics.
<code>environments/action_wrappers.py</code>	Componentwise normalized action clipping, normalized-to-physical affine mapping, and inverse mapping used by audits.
<code>environments/factory.py</code>	Fresh environment construction and exact wrapper nesting order.
<code>projection/cbf_qp_projection.py</code>	Projection parameters, lookahead geometry, CBF rows, physical action bounds, CVXPY/OSQP solve, failure fallback, and immutable result object.
<code>projection/cbf_qp_wrapper.py</code>	Runtime connection between current environment geometry, numerical projector, executed action, and fixed-slot diagnostics in <code>info</code> .
<code>algorithms/ppo/agent.py</code>	Orthogonally initialized diagonal-Gaussian actor and scalar value critic.
<code>algorithms/ppo/ppo_continuous_action.py</code>	CleanRL-style vector rollout, same-step autoreset handling, GAE, PPO-Clip updates, event logging, and checkpoint writing.
<code>algorithms/ppo/action_bound_diagnostics.py</code>	Rollout-level counts, rates, and norms for clipping of unbounded normalized samples.
<code>algorithms/ppo/projection_training.py</code>	Rollout-level projection burden, terminal <code>final_info</code> handling, and fail-fast solver behavior during projected training.
<code>evaluation/evaluate_policy.py</code>	Random/PPO evaluation, complete-episode metrics, checkpoint compatibility, action-bound and projection summaries, CSV output, and optional trajectory recording.
<code>evaluation/evaluate_projection_pair.py</code>	Same-checkpoint, same-seed projection-off/on evaluation and paired episode/summary artifacts.
<code>evaluation/evaluate_layout_suite.py</code>	Evaluation over validated named layouts with checkpoint, method, seed, suite, reward, and projection provenance.
<code>evaluation/layout_suite.py</code>	Layout JSON validation, fixed-capacity padding, duplicate rejection, reset-option creation, and canonical semantic hashing.
<code>evaluation/trajectory_recording.py</code>	<code>evaluation_trajectory_v1</code> NPZ recorder with $T + 1$ state and T transition alignment and exact action-path auditing.
<code>experiments/train_ppo_variant.py</code>	Shared training launcher for the three declared methods and explicit device selection.
<code>experiments/benchmark_training_devices.py</code>	Engineering-only CPU/CUDA throughput and stability benchmark; writes a device decision that is not research evidence.
<code>experiments/calibrate_core_layouts.py</code>	Core-suite paired evaluation, automated audit, summaries, representative trajectories, and calibration PASS marker.
<code>analysis/training_diagnostics.py</code>	TensorBoard event discovery and conversion to episode-level safety and rollout-level clipping/projection tables.
<code>analysis/aggregate_projection_results.py</code>	Protocol loading, metadata-driven episode discovery, completeness checks, checkpoint/method aggregation, pairing, and CSV/JSON evidence tables.

Path	Implemented responsibility
<code>analysis/plot_projection_results.py</code>	PDF figures from audited tables, trajectories, and TensorBoard-derived training diagnostics.
<code>analysis/build_projection_results.py</code>	One-command result build, output-directory safety, audit JSON, and PASS summary.
<code>evaluation/validate_evaluation_time_projection.py</code>	Historical canonical projection regression covering mechanics, pairing, trajectories, stochastic activity, and runtime.
<code>evaluation/validate_pre_experiment_codebase.py</code>	Consolidated pre-experiment gate spanning short training variants, evaluation, artifacts, diagnostics, result build, and manual-review instructions.
<code>evaluation/validate_pre_experiment_artifacts.py</code>	Detailed audit of training, action paths, trajectories, result tables, required figures, and PASS dependencies.
<code>tests/</code>	Lightweight deterministic tests protecting environment, projection, rollout, evaluation, layout, aggregation, diagnostics, and orchestration contracts.
<code>scripts/*.bat</code>	Thin Windows launchers that enter the repository root, invoke a Python module using the active interpreter, and propagate exit status.

3.4 Dependency direction

The environment imports neither PyTorch nor experiment code. Projection depends on environment geometry but not on the actor. The PPO trainer depends on public environment construction and diagnostic interfaces. Evaluation imports the actor only when a PPO checkpoint is requested. Analysis depends on artifact schemas rather than runtime environments. This one-way dependency structure keeps later analysis changes from altering the numerical behavior that generated the evidence.

4 Environment and task contract

4.1 State and fixed-capacity observation

The physical state is

$$p_t = \begin{bmatrix} x_t \\ y_t \end{bmatrix} \in \mathbb{R}^2, \quad \theta_t \in (-\pi, \pi], \quad (2)$$

with goal $p_g \in \mathbb{R}^2$. Obstacle slot i contains center $o_i \in \mathbb{R}^2$, physical radius $r_i \geq 0$, and active mask $m_i \in \{0, 1\}$. The agent has collision radius r_{agent} .

The observation dimension is

$$d_{\text{obs}} = 6 + 5N_{\text{max}}. \quad (3)$$

The global block is

$$[x_t, y_t, \cos \theta_t, \sin \theta_t, x_g - x_t, y_g - y_t]. \quad (4)$$

Each slot contributes

$$[o_{i,x} - x_t, o_{i,y} - y_t, d_{i,t}^{\text{clear}}, r_i, m_i]. \quad (5)$$

Inactive slots are exactly zero. The default $N_{\text{max}} = 3$ produces a 21-dimensional `float32` observation. Capacity determines checkpoint input compatibility; changing the active count alone does not.

4.2 Physical actions and dynamics

The base environment accepts

$$u_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix}, \quad 0 \leq v_t \leq v_{\text{max}}, \quad -\omega_{\text{max}} \leq \omega_t \leq \omega_{\text{max}}. \quad (6)$$

After finite-value validation and defensive clipping, it applies

$$x_{t+1} = x_t + v_t \cos \theta_t \Delta t, \quad (7)$$

$$y_{t+1} = y_t + v_t \sin \theta_t \Delta t, \quad (8)$$

$$\theta_{t+1} = \text{wrap}(\theta_t + \omega_t \Delta t). \quad (9)$$

Position uses the heading at the beginning of the step.

4.3 Collision, success, and episode flags

The signed collision clearance is

$$d_{i,t}^{\text{clear}} = \|p_t - o_i\|_2 - (r_i + r_{\text{agent}}). \quad (10)$$

A collision occurs when the minimum active clearance is non-positive. With no active obstacle, minimum clearance is undefined and represented by NaN; collision remains false. Success occurs when

$$\|p_t - p_g\|_2 \leq r_{\text{goal}}. \quad (11)$$

The Gymnasium flags are

$$\text{terminated} = \text{success} \vee \text{collision}, \quad (12)$$

$$\text{truncated} = (\text{step_count} \geq T_{\text{max}}) \wedge \neg \text{terminated}. \quad (13)$$

The three completed-episode outcomes success, collision, and timeout are therefore mutually exclusive and exhaustive in the training diagnostics.

4.4 Reward

Let $d_t^{\text{goal}} = \|p_t - p_g\|_2$. The implemented reward is

$$r_{t+1} = c_p(d_t^{\text{goal}} - d_{t+1}^{\text{goal}}) - c_u((v_t^{\text{exec}})^2 + (\omega_t^{\text{exec}})^2) - c_{\text{time}} \quad (14)$$

$$+ r_{\text{goal}} \mathbf{1}_{\text{success}, t+1} - c_{\text{collision}} \mathbf{1}_{\text{collision}, t+1} - c_{\text{timeout}} d_{t+1}^{\text{goal}} \mathbf{1}_{\text{timeout}, t+1}. \quad (15)$$

The reward r_{t+1} follows execution of u_t^{exec} ; all terminal indicators and the timeout distance use the resulting state. Only $c_{\text{collision}}$ is intentionally varied by the high-penalty training method. Final evaluation uses collision penalty 10 for every history. Safety conclusions are derived from explicit safety metrics rather than from reward alone.

4.5 Reset and controlled layouts

The default layout uses start $[0, 0]$, heading 0, goal $[4, 0]$, obstacle centers $[1.5, 0.5]$, $[2.5, -0.5]$, and $[3.2, 0.4]$, and radii 0.30, 0.30, and 0.25. The `options` dictionary can override start, heading, goal, centers, radii, and mask while preserving the configured capacity. This interface is used by forced mechanics tests and named layout suites.

Algorithm 1 Environment reset and step contract

Input: Seed, optional controlled layout, or physical action u_t

Output: Gymnasium reset pair or step five-tuple

- 1: On reset, initialize step count, state, goal, fixed-capacity obstacle arrays, and mask.
 - 2: Compute initial distance, fixed-shape observation, and diagnostic mapping.
 - 3: On step, reshape and reject non-finite action values; clip to physical bounds.
 - 4: Save pre-step goal distance and apply the explicit-Euler unicycle update.
 - 5: Increment step count and compute post-step distance, clearance, success, and collision.
 - 6: Compute shaped reward, terminal flags, and timeout-distance penalty when applicable.
 - 7: Encode a finite `float32` observation and return the Gymnasium five-tuple.
-

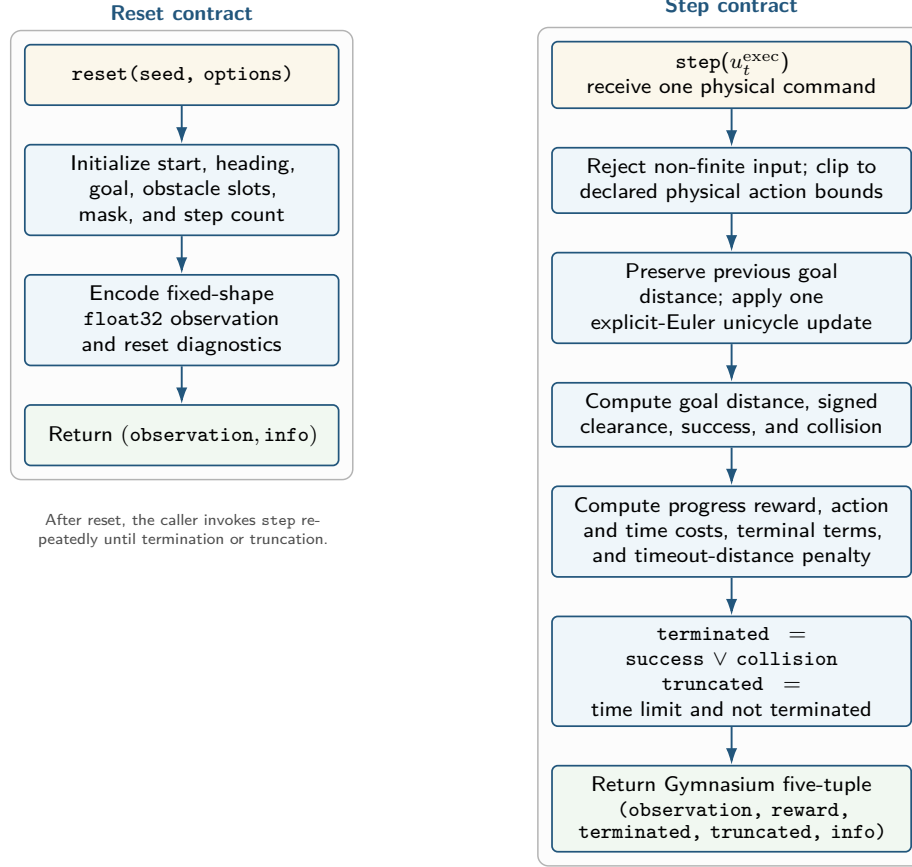


Figure 2: Environment reset and step sequence.

4.6 Environment-level invariants

The base environment accepts physical actions only, clips them defensively, rejects non-finite values, and never computes a policy likelihood. It does not own projection parameters. Conversely, the projection wrapper does not update position, heading, reward, or terminal conditions directly. This separation allows baseline and projected runs to share the identical physical transition and reward implementation.

5 Action interfaces and wrapper topology

5.1 Wrapper nesting

`environments.factory.make_env` constructs wrappers in the following order:

$$\begin{aligned}
 \text{ConstrainedNavigationEnv} &\rightarrow \text{CbfQpProjectionWrapper} \quad (\text{optional}) \\
 &\rightarrow \text{NormalizedActionWrapper} \quad (\text{normally enabled}) \\
 &\rightarrow \text{RecordEpisodeStatistics} \quad (\text{normally enabled}).
 \end{aligned} \tag{16}$$

The outermost wrapper receives the caller action first. Consequently, action flow is the reverse of construction: episode statistics passes the action inward; the normalization wrapper bounds and converts it; the projection wrapper optionally modifies the physical command; and the base environment executes it.

5.2 Four-stage action semantics

The current implementation requires four distinct action variables:

Symbol	Coordinate space	Ownership and meaning
a_t^{sample}	Normalized, unbounded $\mathbb{R}^2$	Sample drawn from the actor’s diagonal Gaussian. PPO stores this exact value and computes both old and new log probabilities at this value.
a_t^{bounded}	Normalized, bounded $[-1, 1]^2$	Componentwise clip of a_t^{sample} performed by <code>NormalizedActionWrapper</code> . It is not a new policy sample.
u_t^{raw}	Physical, bounded	Affine image of a_t^{bounded} . This is the nominal physical command presented to the projection wrapper.
u_t^{exec}	Physical, bounded	Action executed by the base environment. It equals u_t^{raw} when projection is disabled and the projection result otherwise.

The transformations are

$$a_t^{\text{bounded}} = \text{clip}(a_t^{\text{sample}}, -1, 1), \quad (17)$$

$$u_t^{\text{raw}} = g(a_t^{\text{bounded}}) = \begin{bmatrix} \frac{1}{2}v_{\max}(a_{v,t}^{\text{bounded}} + 1) \\ \omega_{\max}a_{\omega,t}^{\text{bounded}} \end{bmatrix}, \quad (18)$$

$$u_t^{\text{exec}} = \begin{cases} u_t^{\text{raw}}, & \text{projection disabled,} \\ \Pi_{\text{safe}}(s_t, u_t^{\text{raw}}), & \text{projection enabled.} \end{cases} \quad (19)$$

A bounded normalized action $[0, 0]$ maps to physical $[0.5v_{\max}, 0]$, not to a stationary action.

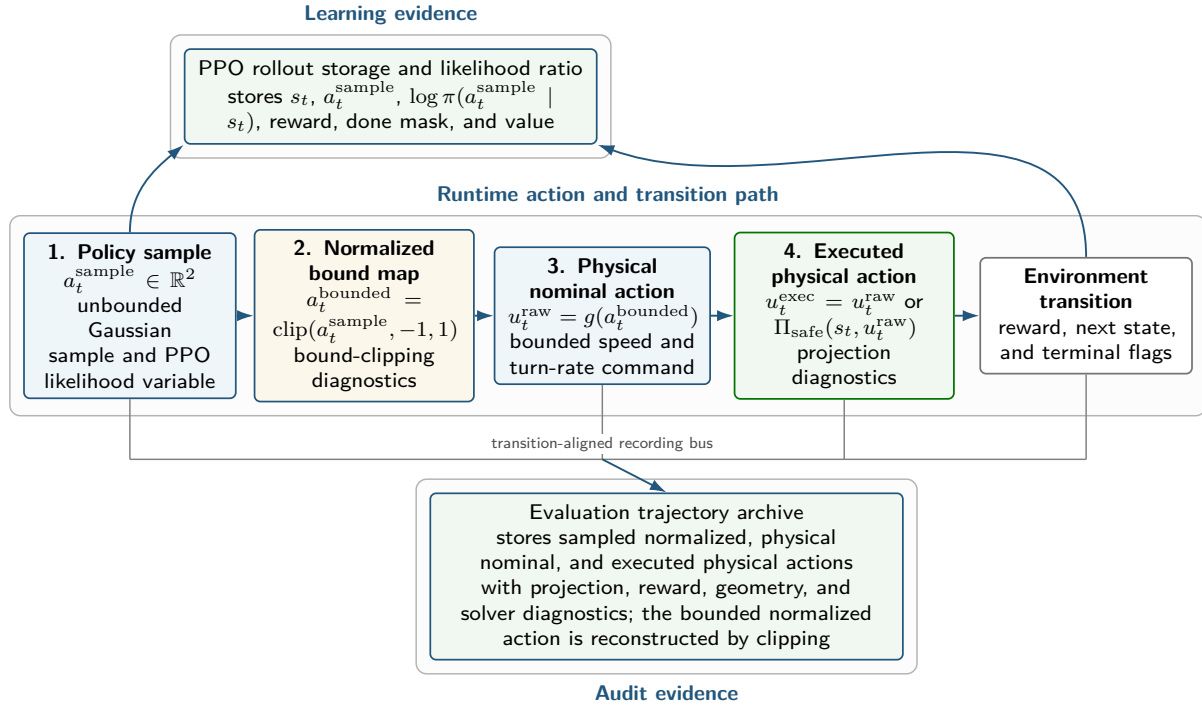


Figure 3: Four-stage action ownership, likelihood semantics, and trajectory recording.

5.3 Normalized action-bound diagnostics

The distance from the sampled normalized action to the bounded normalized action is

$$\Delta a_t^{\text{bound}} = a_t^{\text{sample}} - a_t^{\text{bounded}}, \quad c_t^{\text{bound}} = \|\Delta a_t^{\text{bound}}\|_2. \quad (20)$$

An action is counted as clipped when at least one component differs after clipping. Speed and turn-rate clipping are counted separately. For N transitions, the rollout or episode metrics are

$$f^{\text{bound}} = \frac{1}{N} \sum_{t=1}^N \mathbf{1}_{c_t^{\text{bound}} > 0}, \quad (21)$$

$$f_v^{\text{bound}} = \frac{1}{N} \sum_{t=1}^N \mathbf{1}_{a_{v,t}^{\text{sample}} \notin [-1,1]}, \quad (22)$$

$$f_\omega^{\text{bound}} = \frac{1}{N} \sum_{t=1}^N \mathbf{1}_{a_{\omega,t}^{\text{sample}} \notin [-1,1]}, \quad (23)$$

$$\bar{c}^{\text{bound}} = \frac{1}{N} \sum_{t=1}^N c_t^{\text{bound}}, \quad (24)$$

$$c_{\max}^{\text{bound}} = \max_t c_t^{\text{bound}}. \quad (25)$$

These diagnostics distinguish policy-distribution saturation from CBF-QP intervention. They are computed directly from a_t^{sample} and do not require changing the trajectory schema.

5.4 Action-path invariants

The following identities are audited:

1. $a_t^{\text{bounded}} = \text{clip}(a_t^{\text{sample}}, -1, 1)$ componentwise;
2. $u_t^{\text{raw}} = g(a_t^{\text{bounded}})$ under the environment's declared physical bounds;
3. $\Delta u_t^{\text{proj}} = u_t^{\text{exec}} - u_t^{\text{raw}}$;
4. $\|\Delta u_t^{\text{proj}}\|_2$ agrees with the reported correction norm;
5. projection-disabled transitions satisfy $u_t^{\text{exec}} = u_t^{\text{raw}}$ and zero projection burden;
6. PPO likelihoods remain attached to a_t^{sample} , not a_t^{bounded} , u_t^{raw} , or u_t^{exec} .

6 Predictive action projection

6.1 Lookahead geometry

The local filter uses the point

$$p_L = p + L \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix} \quad (26)$$

and its control Jacobian

$$J_L(\theta) = \begin{bmatrix} \cos \theta & -L \sin \theta \\ \sin \theta & L \cos \theta \end{bmatrix}, \quad \dot{p}_L = J_L(\theta)u. \quad (27)$$

For $L > 0$, both forward speed and turn rate affect the instantaneous lookahead-point velocity. The implementation validates $L \geq 0$.

6.2 Collision and projection radii

Collision detection and projection conservatism are separate contracts:

$$R_i^{\text{collision}} = r_i + r_{\text{agent}}, \quad (28)$$

$$R_i^{\text{projection}} = r_i + r_{\text{agent}} + r_{\text{extra}}. \quad (29)$$

The default $r_{\text{extra}} = 0$ makes the radii equal, but the parameters remain conceptually distinct. Changing extra clearance must not silently change environment collision detection.

6.3 Affine CBF rows

For active obstacle i ,

$$h_i(p_L) = \|p_L - o_i\|_2^2 - (R_i^{\text{projection}})^2, \quad \nabla h_i(p_L) = 2(p_L - o_i). \quad (30)$$

The implementation constructs

$$A_i = \nabla h_i(p_L)^\top J_L(\theta), \quad b_i = -\alpha h_i(p_L), \quad (31)$$

so that the softened row is

$$A_i u + \xi_i \geq b_i, \quad \xi_i \geq 0. \quad (32)$$

Only active obstacle slots produce rows. The stored active indices retain the mapping back to fixed-capacity obstacle slots.

6.4 Projection QP

For M active constraints, the numerical problem is

$$\begin{aligned} & \underset{u \in \mathbb{R}^2, \xi \in \mathbb{R}^M}{\text{minimize}} && \frac{1}{2} \|u - u^{\text{raw}}\|_2^2 + \frac{\rho}{2} \|\xi\|_2^2 \\ & \text{subject to} && 0 \leq u_1 \leq v_{\max}, \\ & && -\omega_{\max} \leq u_2 \leq \omega_{\max}, \\ & && A_{\text{cbf}} u + \xi \geq b_{\text{cbf}}, \\ & && \xi \geq 0. \end{aligned} \quad (33)$$

CVXPY constructs the problem and OSQP is the default solver [2, 8]. Warm start is enabled. Successful statuses are `optimal` and `optimal_inaccurate`; numerical finiteness is checked after the solve.

6.5 Projection result contract

`ProjectionResult` contains:

- physical raw and executed actions;
- correction vector and Euclidean norm;
- intervention flag using a configurable tolerance;
- active-obstacle slack vector, sum, and maximum;
- active constraint count;
- solver status and Boolean success;
- optional objective value.

The wrapper expands active slack values into a fixed-capacity slot-aligned vector before writing `info`. Inactive slots are zero on successful solves.

6.6 Special paths and failure semantics

Path	Executed action and status	Slack semantics
No active constraints	Clipped physical raw action; success true; status <code>no_active_constraints</code> .	Empty numerical slack, expanded to fixed-slot zeros by the wrapper.
Valid optimal solve	Finite bounded numerical solution; <code>success=True</code> ; solver status preserved.	Finite nonnegative active slack; inactive slots zero; sum and maximum computed from active values.
CVXPY unavailable	Stationary action $[0, 0]$; <code>success=False</code> ; status <code>cvxpy_not_installed</code> .	Every active slack is NaN; sum and maximum are NaN.
Solver exception	Stationary action $[0, 0]$; <code>success=False</code> ; status <code>solver_error</code> .	Unknown active slack represented by NaN.
Nonoptimal or missing solution	Stationary action $[0, 0]$; <code>success=False</code> ; original status preserved.	Unknown active slack represented by NaN.
Non-finite returned solution	Stationary action $[0, 0]$; <code>success=False</code> ; status <code>nonfinite_solution</code> .	Unknown active slack represented by NaN.

A failure is never encoded as successful zero slack. This distinction propagates to episode CSVs, trajectories, training fail-fast behavior, and result-build rejection.

Algorithm 2 Physical CBF-QP action projection

Input: State geometry, obstacle arrays and mask, agent radius, u^{raw} , projection parameters

Output: u^{exec} and `ProjectionResult`

- 1: Validate finite shapes and parameters.
 - 2: Select active obstacle indices and compute p_L and J_L .
 - 3: **for** each active obstacle **do**
 - 4: Compute projection radius, h_i , ∇h_i , A_i , and b_i .
 - 5: **end for**
 - 6: **if** no active constraints **then**
 - 7: Return clipped u^{raw} , success, zero slack, and `no_active_constraints`.
 - 8: **else**
 - 9: Construct the bounded softened QP and solve with configured solver.
 - 10: **if** status is accepted and action/slack are finite **then**
 - 11: Clip numerical action defensively and clamp slack below at zero.
 - 12: Return success and computed correction/slack diagnostics.
 - 13: **else**
 - 14: Return stationary $[0, 0]$, failure status, and unknown active slack.
 - 15: **end if**
 - 16: **end if**
-

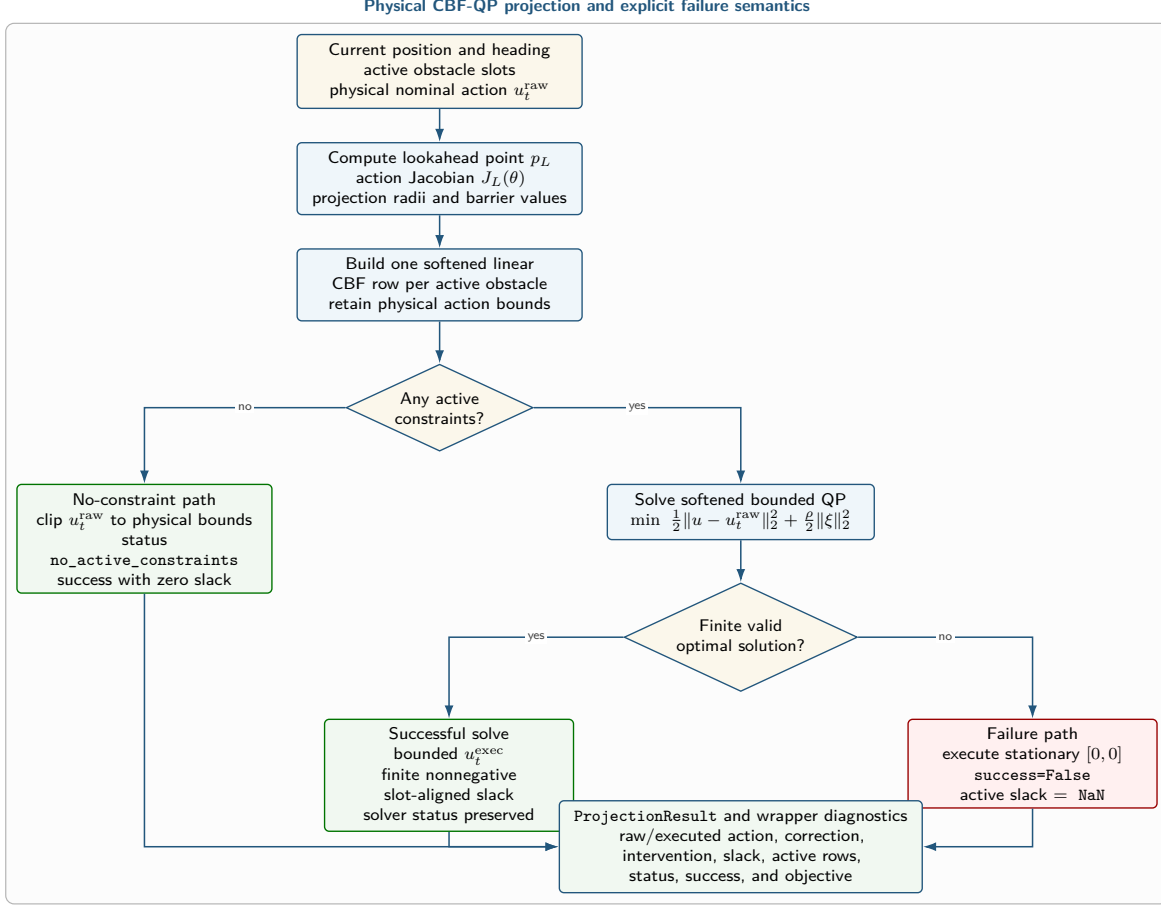


Figure 4: Projection geometry, QP solve, and explicit failure path.

6.7 Interpretive meaning of diagnostics

Projection intervention is defined by

$$I_t^{\text{proj}} = \mathbf{1}_{\|u_t^{\text{exec}} - u_t^{\text{raw}}\|_2 > \varepsilon_{\text{corr}}} \quad (34)$$

A zero intervention under an enabled filter means the nominal physical action was already accepted to numerical tolerance. A high intervention rate with low collision rate indicates possible filter dependence. Positive slack indicates local constraint relaxation. A collision may still occur despite projection because the condition is local, discrete-time, softened, and not a global route planner.

7 PPO implementation and safeguarded-environment semantics

7.1 Actor-critic architecture

The actor and critic are independent two-hidden-layer multilayer perceptrons with 64 tanh units per layer and orthogonal initialization. The actor outputs the mean of a diagonal Normal distribution and owns a learned state-independent log-standard-deviation parameter. The critic outputs one scalar value estimate. The actor distribution is not tanh-squashed; therefore a_t^{sample} may exceed $[-1, 1]^2$ and the separate action-bound diagnostics are scientifically relevant.

7.2 Effective projected environment

For a normalized sampled action a , define the deterministic action interface

$$F(s, a) = \Pi_{\text{safe}}(s, g(\text{clip}(a, -1, 1))) \quad (35)$$

when projection is enabled, with the projection map replaced by the identity when disabled. The effective reward and transition are

$$r_F(s, a) = r(s, F(s, a)), \quad (36)$$

$$P_F(s' \mid s, a) = P(s' \mid s, F(s, a)). \quad (37)$$

PPO samples and scores a_t^{sample} while rewards and successor states arise from $F(s_t, a_t^{\text{sample}})$. This is an empirical safeguarded-environment formulation. Projection does not make the data off-policy; the action was generated by the current rollout policy, and the projection is a deterministic part of the effective environment. The implementation nevertheless makes no claim that PPO’s clipped surrogate is an unbiased corrected projected-policy-gradient estimator [3].

7.3 Vector rollout and same-step autoreset

Training uses `gym.vector.SyncVectorEnv` with `AutoresetMode.SAME_STEP`. A terminal vector step returns the reset observation for continued collection and preserves terminal episode information in `final_info`. Projection diagnostics can also appear in `final_info`; the training accumulator consumes both ordinary vector information and terminal information so that every executed transition is counted exactly once.

The rollout tensors are

$$\text{obs} : (T, N_{\text{env}}, d_{\text{obs}}), \quad (38)$$

$$\text{actions_raw} : (T, N_{\text{env}}, 2), \quad (39)$$

$$\text{logprobs, rewards, dones, values} : (T, N_{\text{env}}). \quad (40)$$

The PPO buffer intentionally stores a_t^{sample} rather than u_t^{raw} or u_t^{exec} .

7.4 GAE and PPO-Clip

For done mask $m_t = 1 - d_t$, the implementation computes

$$\delta_t = r_{t+1} + \gamma m_t V(s_{t+1}) - V(s_t), \quad (41)$$

$$\hat{A}_t = \delta_t + \gamma \lambda m_t \hat{A}_{t+1}, \quad \hat{R}_t = \hat{A}_t + V(s_t), \quad (42)$$

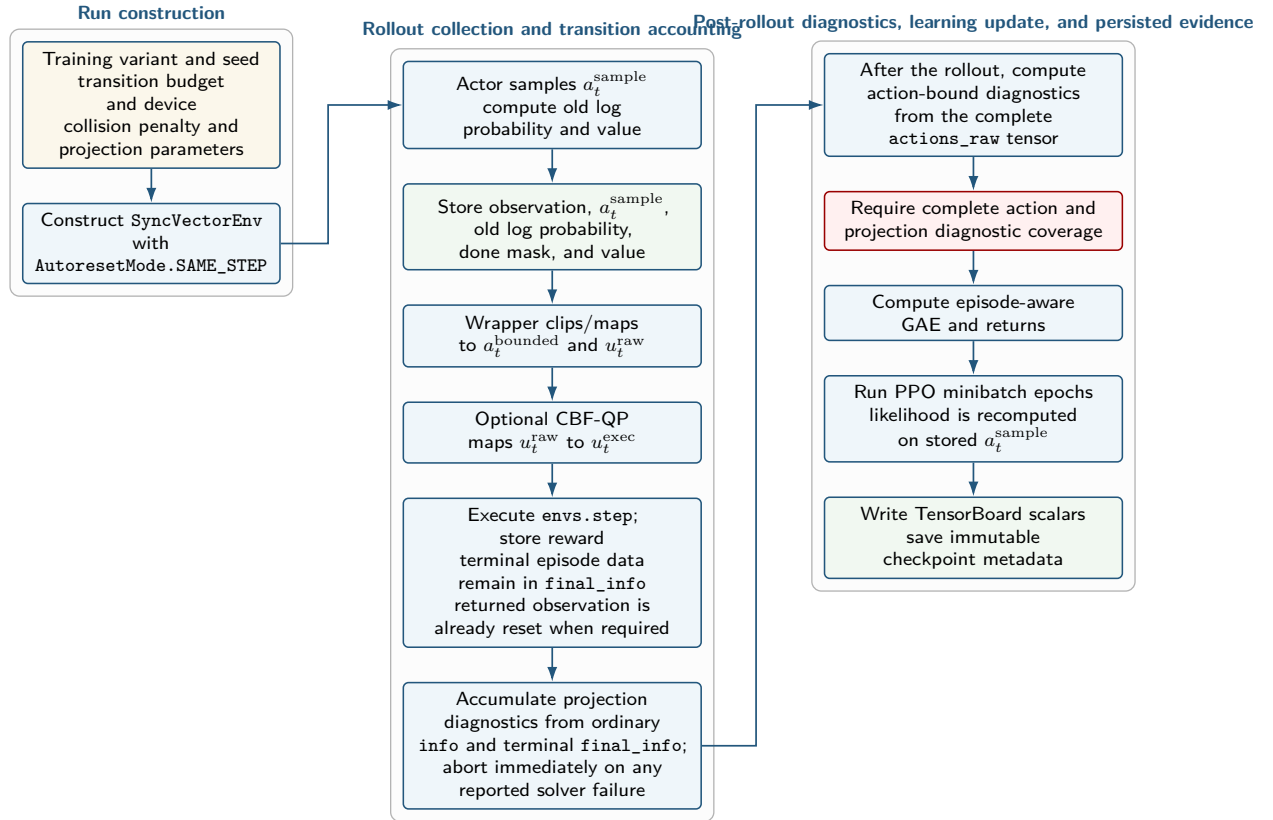
backward over the rollout. The likelihood ratio is

$$r_t(\theta) = \exp\left(\log \pi_{\theta}(a_t^{\text{sample}} \mid s_t) - \log \pi_{\theta_{\text{old}}}(a_t^{\text{sample}} \mid s_t)\right), \quad (43)$$

and the clipped policy loss uses the usual minimum of the unclipped and clipped surrogate [7, 6]. The value function uses the configured clipped value loss; gradients are norm-clipped.

Algorithm 3 PPO rollout with action-bound and projection diagnostics**Input:** Current actor-critic, vector environment, training variant, rollout length T **Output:** Rollout tensors and one set of rollout-level diagnostics

- 1: **for** $t = 0, \dots, T - 1$ **do**
- 2: Store current observation and episode-boundary mask.
- 3: Sample a_t^{sample} ; compute old log probability and value estimate.
- 4: Send a_t^{sample} to the vector environment.
- 5: The wrapper computes a_t^{bounded} , u_t^{raw} , and optional u_t^{exec} .
- 6: The environment advances under u_t^{exec} and returns reward, flags, and diagnostics.
- 7: Preserve terminal episode and projection data from `final_info`.
- 8: Accumulate projection diagnostics and abort immediately on any reported solve failure.
- 9: **end for**
- 10: Compute action-bound diagnostics from the complete raw action tensor.
- 11: Require action-bound and, when enabled, projection diagnostics for every transition.
- 12: Compute GAE and returns using episode-boundary masks.
- 13: Perform PPO minibatch epochs using stored a_t^{sample} .
- 14: Write PPO, task, action-bound, and projection events; save checkpoint when requested.

**Figure 5:** Implemented PPO training path with same-step autoreset and separate clipping/projection diagnostics.

7.5 Training diagnostics

At completed episodes, the trainer writes return, length, success, collision, timeout, and terminal clearance when finite. At every rollout update, it writes normalized action-bound metrics. When projection is enabled, it additionally writes transition and intervention counts, intervention frequency, mean and maximum correction, mean summed slack, maximum slack, and solver-failure count.

The projection accumulator validates finite values and includes terminal `final_info`. A solver failure raises immediately; it is not allowed to enter a projected-training checkpoint as an ordinary transition. After each rollout, transition counts are checked against the full batch size.

7.6 PPO selection relative to SAC

PPO is not claimed to be universally superior to SAC. The implemented choice is advantageous for this study because the fresh rollout, raw sampled action, old log probability, and resulting projected transition are visible in one compact path. SAC could be implemented coherently by storing the raw normalized action in replay and learning the action value of the effective projected environment, but it would introduce replay-distribution semantics, twin critics, targets, entropy-temperature behavior, and actor gradients through a learned Q surface whose raw-action geometry is many-to-one under clipping and projection. PPO therefore reduces attribution complexity even though CleanRL makes the source-code length of the two algorithms comparable.

7.7 Checkpoint contract

A saved checkpoint contains:

- `agent_state_dict`;
- the complete resolved CLI argument mapping;
- run name and final global step;
- observation and action shapes;
- flattened observation and action dimensions;
- the actual selected device in the arguments.

Evaluation validates checkpoint dimensions against the requested environment capacity. Historical checkpoints remain useful as regression fixtures only when their recorded semantics are respected.

8 Training variants, configuration, and campaign controls

8.1 Shared launcher

`experiments/train_ppo_variant.py` is the single orchestration point for the declared training methods. It constructs the exact Python command, refuses an existing checkpoint path, creates only the parent directory, invokes the active interpreter from the repository root, and verifies that the checkpoint exists when the child process completes.

The launcher supports explicit `auto`, `cpu`, or `cuda` device selection. The default training budget is 51,200 transitions. The fixed launch configuration is:

Setting	Value
Vector environments	4
Steps per environment per rollout	256
Batch size	1,024
Minibatches	8
PPO epochs per rollout	4
Maximum episode steps	200
Obstacle capacity / active count	3 / 3
Learning rate	3×10^{-4} with linear annealing
Discount / GAE parameter	$\gamma = 0.99, \lambda = 0.95$
PPO clip coefficient	0.2
Value coefficient / entropy coefficient	0.5 / 0
Maximum gradient norm	0.5

8.2 Training methods

Method label	Collision penalty	Training projection	Controlled difference
ppo_baseline	10	disabled	Reference PPO interaction.
ppo_high_penalty	50	disabled	Reward shaping changes only the collision penalty.
ppo_train_projection	10	enabled	Policy and critic learn from projected environment transitions while PPO scores raw samples.

8.3 Protocol-controlled evaluation matrix

The analysis protocol schema separates training history from runtime projection mode. Both frozen completed-study protocols require the following six cells:

Training method	Projection disabled	Projection enabled	Design interpretation
PPO baseline	required	required	Compares runtime controllers for the same reference checkpoint.
PPO high penalty	required	required	Separates reward shaping from deployment-time filtering.
PPO trained with projection	required	required	Supports filter-on performance and filter-dependence analysis.

The final training configuration uses seeds 1, 2, 3, 4, 5, 51,200 transitions per run, and CPU execution, yielding 15 final checkpoints. The frozen evaluation configurations are:

- `experiments/fixed_training_geometry_analysis_protocol.json`: 100 stochastic episodes per checkpoint and runtime mode, seeds 10000–10099, for 3,000 episodes in total;
- `experiments/projection_analysis_protocol.json`: one deterministic actor-mean execution per checkpoint, runtime mode, and each of 24 prespecified layouts, seeds 1000–1023, for 720 executions in total.

Both use evaluation collision penalty 10. Fixed geometry and transfer remain separate evidence families; their comparison changes both geometry and action selection.

8.4 Development protocol

`experiments/development_projection_analysis_protocol.json` uses a small development layout suite and relaxed completeness requirements to exercise artifact discovery, pairing, aggregation, and plotting. Its values are software-validation fixtures rather than commitments for an experimental campaign.

8.5 Training device benchmark

`experiments/benchmark_training_devices.py` compares CPU and CUDA on the baseline and projection-training paths using a configurable diagnostic seed and short transition budget; the command defaults are seed 9902 and 10,240 transitions. The weighted time is

$$T_d = 2T_{d,\text{baseline}} + T_{d,\text{projection}}, \quad (44)$$

representing two baseline-like methods and one projected method. CUDA is selected only when all CUDA conditions are stable and weighted runtime is at least ten percent lower than CPU. The recorded CPU selection is an engineering input to the frozen final protocol and the benchmark returns are excluded from research conclusions.

8.6 Independent replication unit

The independent empirical replicate is a training run. One retained final checkpoint represents each run; environment episodes and layouts are repeated observations within that run. There are five runs

per training history, not 100 or 24 independent learned policies. Analysis retains individual run values and reports means, sample standard deviations, and paired deltas separately for each evaluation suite. These summaries support descriptive, retrospective interpretation.

9 Evaluation architecture

9.1 Evaluation modes

`evaluation.evaluate_policy` evaluates random actions or a PPO checkpoint for complete episodes. A PPO checkpoint can be evaluated using:

Deterministic actor mean. Supports reproducible task and transfer comparisons, including named-layout evaluation.

Stochastic policy samples. Exposes the safety and clipping behavior of the action distribution and is used by active-path diagnostics.

Evaluation constructs the same wrapper stack as training, holds network parameters fixed, validates observations and rewards for finiteness, and enforces a step guard in addition to environment termination and truncation.

9.2 Episode result schema

Each `EpisodeResult` records:

- identity: policy, checkpoint, episode index, and evaluation seed;
- task: return, length, success, collision, terminated, truncated, final goal distance, and minimum signed clearance;
- action bounds: clipping counts and rates for any component, speed, and turn rate, plus mean and maximum clipping norm;
- projection: enabled flag, intervention count and rate, mean and maximum correction norm, mean summed slack, maximum individual slack, and solver-failure count.

Projection parameter metadata is appended to each CSV row. When projection is disabled, projection fields retain a stable zero-valued identity convention. When any solve fails, episode slack aggregates remain `NaN`; failure cannot be averaged into valid zero slack.

9.3 Single-policy evaluation algorithm

Algorithm 4 Complete-episode policy evaluation

Input: Environment factory, action provider, episode seeds, projection mode, optional reset layout

Output: Episode rows and optional trajectory records

```

1: for each requested episode do
2:   Reset with its declared seed and optional controlled layout.
3:   Initialize task, action-bound, projection, and trajectory accumulators.
4:   while episode is not terminated or truncated do
5:     Obtain a random action, actor mean, or stochastic actor sample.
6:     Update action-bound diagnostics from the normalized action.
7:     Reconstruct  $u^{\text{raw}}$  for trajectory auditing when recording is enabled.
8:     Step the wrapped environment and append transition diagnostics.
9:     Update task, clearance, projection, and solver-failure summaries.
10:  end while
11:  Require action-bound transition count to equal episode length.
12:  Finalize rates and means; preserve unknown slack if a failure occurred.
13: end for
14: Write one CSV row per episode and, if requested, one aligned NPZ archive.
  
```

9.4 Same-checkpoint paired evaluation

`evaluation.evaluate_projection_pair` loads one checkpoint and runs projection-disabled and projection-enabled evaluation under one shared configuration. It hashes the checkpoint before and after evaluation. Episode keys are aligned by (`episode`, `seed`) and duplicates are rejected.

For output prefix `<prefix>`, the workflow writes:

```

<prefix>_projection_disabled.csv
<prefix>_projection_enabled.csv
<prefix>_projection_disabled_trajectories.npz
<prefix>_projection_enabled_trajectories.npz
<prefix>_paired_episodes.csv
<prefix>_paired_summary.csv
  
```

The paired tables include task and action-bound values for both modes, projection burden for the enabled mode, and enabled-minus-disabled deltas. For stochastic evaluation, both modes start each episode with the same environment seed and policy random state; trajectories are allowed to diverge after the first changed executed action.

9.5 Named-layout evaluation

`evaluation.evaluate_layout_suite` loads a validated JSON suite and evaluates every declared layout and repeat. Each row adds:

- method and training seed;
- training collision penalty and training-projection flag;
- checkpoint path and SHA-256;
- layout schema version, suite ID, and semantic SHA-256;
- layout ID, repeat, and evaluation seed;
- evaluation collision penalty, policy mode, and projection mode.

The canonical pairing key is

$$(\text{layout_id}, \text{layout_repeat}, \text{evaluation_seed}). \quad (45)$$

Existing CSV or NPZ output paths are refused unless explicit overwrite is requested.

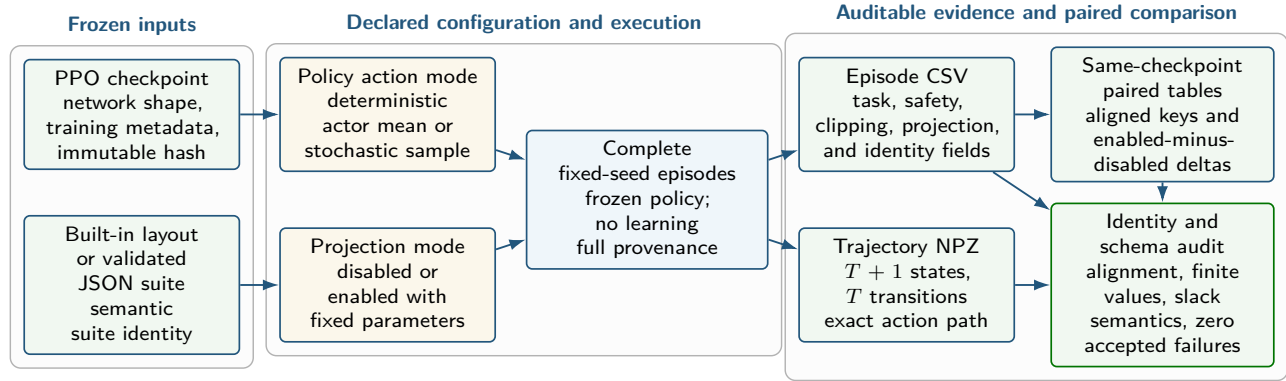


Figure 6: Evaluation sources, modes, paired outputs, and audit path.

9.6 Metric interpretation

The primary episode outcome metrics are not interchangeable:

- collision OFF and success ON records completion only in the ON execution;
- collision OFF and timeout ON records a different failure outcome without ON completion;
- success OFF and timeout or collision ON records completion only in the OFF execution;
- zero intervention under an enabled filter is valid noninterference;
- persistent intervention describes correction burden; OFF/ON performance is also needed to assess runtime dependence;
- action-bound clipping indicates saturation of the Gaussian policy before the physical safety layer.

These are matched terminal-outcome correspondences, not physical conversions or causal counterfactuals. Later states and nominal actions may diverge after the first different executed action. Return is interpreted alongside success, collision, timeout, clearance, and burden.

10 Layout suites, calibration, and benchmark identity

10.1 Layout suite schema

The layout suite schema identifier is `navigation_layout_suite_v1`. A suite defines:

- nonempty suite ID;
- obstacle capacity, agent radius, and goal radius;
- one or more unique layout IDs;
- start, heading, goal, and a list of positive-radius obstacles per layout.

The loader validates finite values, capacity, duplicate IDs, start/goal separation, and absence of start or goal collision. Each layout is padded to the suite capacity and converted to fresh arrays for `reset(options=...)`.

10.2 Semantic hashing

Suite identity is computed from parsed JSON serialized with sorted keys and compact separators. It is therefore independent of LF versus CRLF line endings and superficial whitespace. Exact file-byte

hashing remains available for ordinary files, but scientific layout identity is semantic.

10.3 Development suite

The development suite exists to exercise layout loading, provenance, trajectory recording, aggregation, and plotting with low computational cost. It may be changed when software development requires it; it is not a final benchmark.

10.4 Frozen core suite

The core suite contains 24 deterministic layouts in four families:

Family	Count
Control and noninterference	4
Single-obstacle deviations	6
Two-obstacle interactions	8
Three-obstacle slaloms	6

It holds start, heading, goal, dynamics, reward, agent footprint, goal radius, and episode limit fixed while changing obstacle geometry and active count. Non-symmetric cases have reflected counterparts. The suite is a secondary deterministic transfer and robustness benchmark, not a sample from a broad task distribution.

10.5 Calibration procedure

Calibration used a historical regression checkpoint only as a probe and was completed before final training results were inspected. The automated procedure was allowed to reject malformed, uniformly trivial, uniformly impossible, nonintervening, constantly intervened, or unstable geometry. It was not allowed to require a favorable projection result.

Algorithm 5 Core-suite calibration and freeze

Input: Candidate suite, historical probe checkpoint, fixed projection and evaluation configuration

Output: Automated audit, representative trajectories, and accept/revise record

- 1: Evaluate every layout with projection disabled and enabled.
 - 2: Verify checkpoint and suite identities and aligned layout keys.
 - 3: Audit finite metrics, action maps, trajectory alignment, intervention diversity, and zero solver failures.
 - 4: Generate family summaries, layout-level comparisons, and predeclared trajectory panels.
 - 5: Conduct focused manual review of geometry, controls, diversity, and representative behavior.
 - 6: Accept unchanged or revise only for a documented policy-independent defect.
 - 7: Freeze the semantic suite identity and preserve the complete raw calibration bundle.
-

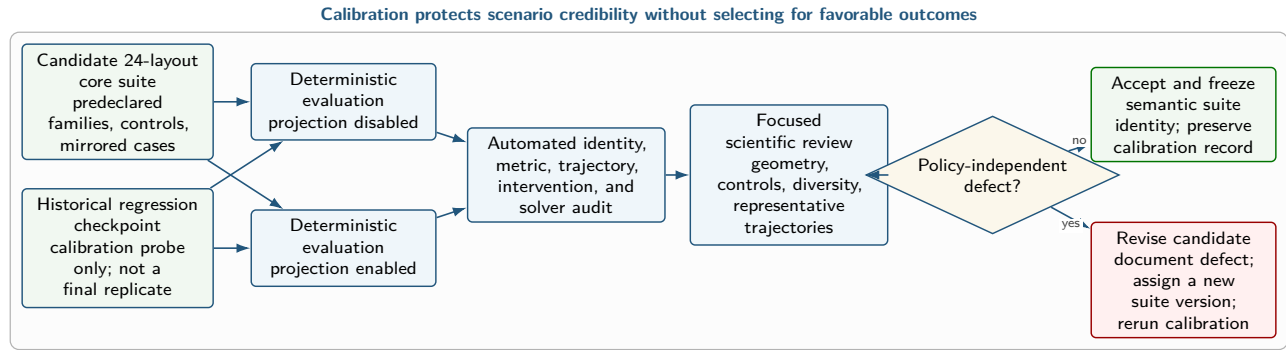


Figure 7: Core-suite calibration and freeze workflow.

10.6 Calibration record boundary

The calibration workflow must preserve the accepted or revised suite identity, input hashes, probe role, acceptance criteria, automated audit, focused review, and freeze decision in a separate research record. This implementation design defines the procedure and the software interfaces; it does not reproduce calibration outcome counts or assert the status of a particular calibration run. Any geometry change requires a new suite identity and a new calibration record.

10.7 Evaluation-level separation

The software supports two complementary evaluation levels that must remain distinct in a campaign protocol:

- repeated stochastic-policy evaluation on the fixed training geometry, which measures the behavior of the learned action distribution on the task used for training;
- deterministic actor-mean evaluation on a frozen named-layout suite, which measures transfer and robustness to changed obstacle geometry.

These levels answer different questions and must not be pooled into one undifferentiated headline metric. Exact episode counts, seed ranges, required projection modes, and aggregation outputs are frozen in protocol artifacts rather than inferred by analysis code.

11 Artifact contracts and provenance

11.1 Artifact lifecycle

The working directory separates local raw evidence from versioned curated outputs and documentation:

```

runs/checkpoints/ raw model checkpoints
runs/evaluation/ raw episode CSV and trajectory NPZ evidence
runs/validation/ validation, smoke, and device-benchmark evidence
runs/calibration/ complete calibration evidence
results/tables/ curated generated tables
results/figures/ curated generated PDF figures
docs/records/ concise historical and freeze records
  
```

All `runs/` contents, including final checkpoints, TensorBoard events, CSV files, and trajectories, are excluded from Git and preserved in local archives. The repository retains source, protocols, curated tables, and analysis tools. Raw-artifact analyses require the archive or newly generated outputs. Analysis builds refuse existing result directories.

11.2 Checkpoint identity

Checkpoint compatibility is determined by stored observation and action shapes/dimensions, not by filename. SHA-256 is recorded in evaluation artifacts. Paired evaluation verifies that the file hash is unchanged before and after the run. A checkpoint’s training metadata includes method, collision penalty, projection flag, seed, global step, and CLI configuration.

11.3 TensorBoard event contract

The event stream includes the following groups:

Tag or group	Meaning
charts/episodic_return, charts/episodic_length	One event per completed episode.
safety/success, safety/collision, safety/timeout	Mutually exclusive and exhaustive completed-episode outcomes.
safety/final_obstacle_clearance	Terminal clearance when finite; absent for undefined obstacle-free cases.
action_bounds/*	Rollout-level normalized action clipping frequency, component rates, mean norm, and maximum norm.
projection/*	Rollout-level transition/intervention counts, intervention frequency, mean/maximum correction, mean summed/maximum slack, and solver-failure count when training projection is enabled.
losses/*	Policy loss, value loss, entropy, approximate KL diagnostics, clip fraction, and explained variance.
charts/learning_rate, charts/SPS	Optimizer schedule and throughput.

Final result protocols require complete training evidence for each checkpoint. Projection-trained runs must contain both mean and maximum burden metrics and zero solver failures.

11.4 Episode CSV contract

One row represents one complete episode. The base row combines `EpisodeResult`, projection parameters, and optional workflow metadata. Layout and paired evaluators add method, training seed, checkpoint hash, suite identity, evaluation mode, reward configuration, projection mode, and pairing ranges. Projection-disabled rows retain the same scientific schema rather than dropping projection columns.

11.5 Trajectory archive contract

The archive schema is `evaluation_trajectory_v1`. It contains non-object NumPy arrays and must load with `allow_pickle=False`. For T transitions:

$$\text{state arrays have length } T + 1, \quad \text{transition arrays have length } T. \quad (46)$$

Index t aligns

$$s_t, a_t^{\text{sample}}, u_t^{\text{raw}}, u_t^{\text{exec}}, r_{t+1}, s_{t+1}. \quad (47)$$

The archive stores static geometry, state trajectories, raw normalized samples, raw and executed physical actions, corrections, rewards, flags, fixed-slot slack, solver status, and projection configuration. Projection-disabled transitions use status `disabled`, which distinguishes the identity path from a successful zero-correction QP solve.

The recorder enforces $T + 1$ versus T alignment, finite actions, raw-physical reconstruction, active-count agreement, correction identities, slack consistency, failure-time unknown slack, and duplicate episode/seed rejection before serialization.

11.6 Analysis protocol contract

The schema `projection_analysis_protocol_v1` declares:

- study ID and referenced layout suite;
- expected training seeds and repeats;
- evaluation policy mode and collision penalty;
- projection parameters;
- representative layout;
- methods and required projection modes;
- whether complete training and trajectory evidence is mandatory;
- training-episode rolling-window length when present.

The protocol deliberately does not hardcode generated checkpoint paths, TensorBoard directories, CSV filenames, or NPZ filenames. Those are discovered from internal metadata.

11.7 Generated output contract

The result build may generate:

```
evaluation_episode_results.csv
checkpoint_summary.csv
method_summary.csv
paired_projection_deltas.csv
paired_projection_summary.csv
training_scalar_events.csv
training_episode_diagnostics.csv
training_rollout_diagnostics.csv
training_curve_points.csv
result_build_audit.json
PDF figures and a PASS summary
```

12 Analysis and statistical aggregation

12.1 Metadata-driven discovery

`analysis.aggregate_projection_results` reads candidate CSV files and accepts them by schema and internal metadata, not by generated filename. It rejects missing columns, duplicate episode keys, incomplete methods/seeds/modes/layouts, mismatched checkpoint or suite identities, different evaluation key sets across seeds, non-finite task values, invalid diagnostics, and nonzero solver failures in accepted final evidence.

12.2 Checkpoint-first hierarchy

Let $y_{m,s,l}$ be a metric for method m , training seed s , and layout case l . The implemented hierarchy is

$$\bar{y}_{m,s} = \frac{1}{L} \sum_{l=1}^L y_{m,s,l}, \quad (48)$$

$$\bar{y}_m = \frac{1}{S} \sum_{s=1}^S \bar{y}_{m,s}. \quad (49)$$

Layouts are averaged within the final checkpoint representing each run before aggregation across the five runs. The primary suite similarly averages stochastic episodes within each run; the suites are not pooled. Standard deviations reported at the method level describe variation across training seeds, not variation across layouts treated as independent policies.

12.3 Paired projection deltas

Projection-disabled and projection-enabled rows are paired within the same checkpoint and layout key before seed-level aggregation. For a metric y ,

$$\Delta y_{m,s,l} = y_{m,s,l}^{\text{enabled}} - y_{m,s,l}^{\text{disabled}}. \quad (50)$$

The paired summary reports run-level and history-level deltas while retaining individual run values. It compares the same learned parameters under two runtime controllers. It does not identify a projector-only causal effect, a common later state sequence, or a physical conversion between matched outcomes.

12.4 Training diagnostic exports

`analysis.training_diagnostics` uses schema `training_diagnostics_v1`. It discovers TensorBoard events associated with checkpoints and builds:

Raw scalar table. One row per event tag, step, wall time, and checkpoint/run identity.

Episode diagnostics. Aligned return, length, success, collision, timeout, optional clearance, completed-episode index, cumulative outcome counts, and rolling rates.

Rollout diagnostics. Aligned action-bound metrics for every run and projection metrics for projection-enabled training.

Curve points. Aggregated training curves with explicit method and seed provenance.

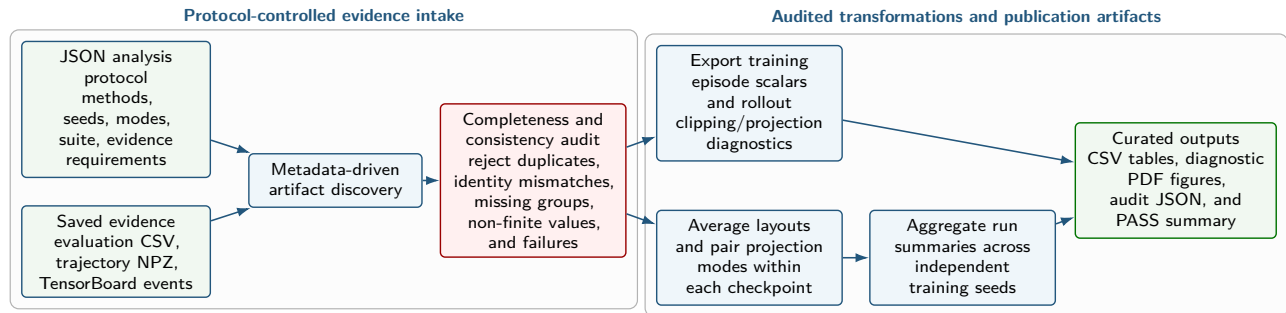
Episode outcomes are validated as binary, mutually exclusive, and exhaustive. Rollout tags must have aligned global steps. Projection transition and intervention counts are checked against expected batch-level behavior when required.

12.5 Scientific diagnostic figures

The plotter generates evaluation return, success, collision, clearance, action-bound clipping, projection intervention, correction, and slack figures; training return and length; cumulative collisions/successes/-timeouts; rolling safety rates; action-bound clipping curves; projection-burden curves; and predeclared trajectory panels when evidence is available. It uses a noninteractive Matplotlib backend and generates figures only from saved, audited artifacts.

Algorithm 6 Audited result build**Input:** Frozen analysis protocol, evaluation directory, optional runs directory, new output directory**Output:** Audited tables, figures, metadata, and PASS summary

- 1: Refuse a nonempty output directory.
- 2: Load and validate the protocol and referenced semantic layout suite.
- 3: Discover evaluation CSV and trajectory NPZ artifacts from internal metadata.
- 4: Validate complete method/seed/mode/layout coverage and scientific identities.
- 5: Discover checkpoint-linked TensorBoard events and build training diagnostics.
- 6: Aggregate layouts within checkpoints and pair projection modes.
- 7: Aggregate run summaries across independent training seeds.
- 8: Write CSV tables, diagnostic figures, and `result_build_audit.json`.
- 9: Validate required artifacts and completeness conditions.
- 10: Write the PASS summary only after every required phase succeeds.

**Figure 8:** Artifact discovery, audit, training exports, hierarchical aggregation, and scientific diagnostic output.

13 Orchestration and validation architecture

13.1 Python orchestration and Windows launchers

Substantive workflow logic lives in Python. Batch files are constrained to entering the repository root, invoking a Python module with explicit arguments, and propagating its exit code. Tests enforce this thin-launcher policy. This avoids hidden shell logic, keeps the active interpreter visible, and makes workflows callable from validation modules without duplicating commands.

Destructive operations are limited to reserved validation or benchmark trees. Orchestration functions validate cleanup targets before replacement and refuse existing final checkpoints or result directories by default.

13.2 Validation layers

The repository uses several layers with different purposes:

Layer	Entry point	Contract
Lightweight tests	<code>python -m pytest -q -rs</code>	Protect deterministic environment, projection, rollout, evaluation, layout, diagnostics, aggregation, and orchestration mechanics. The pass count is not fixed; failures, errors, and unexpected skips are unacceptable in the intended environment.
Evaluation-time projection regression	<code>scripts/validate_evaluation_time_projection.bat</code>	Historical regression using the known baseline fixture; covers compilation, tests, checkpoint compatibility, paired deterministic/stochastic evaluation, CSV/NPZ audit, and projection runtime.
Consolidated pre-experiment validation	<code>scripts/validate_pre_experiment_codebase.bat</code>	Experiment-readiness gate: short training for all variants, checkpoint and event audits, action-bound and projection diagnostics, development-suite evaluation, trajectories, result aggregation, figures, and manual-review instructions.
Artifact audit	<code>evaluation/validate_pre_experiment_artifacts.py</code>	Validates generated training/evaluation/result evidence, including action identities, clipping metrics, slack semantics, required artifacts, and PASS dependencies.
Device benchmark	<code>scripts/benchmark_training_devices.bat</code>	Engineering-only runtime/stability comparison after pre-experiment PASS; does not contribute outcomes to the paper.
Core calibration	<code>scripts/calibrate_core_layouts.bat</code>	Automated technical PASS followed by an explicit scientific accept/revise decision and immutable freeze record.
Manual figure inspection	<code>scripts/inspect_pre_experiment_validation.bat</code>	Focused review for blank, clipped, mislabeled, or table-inconsistent figures; not a subjective performance gate.

13.3 Completed-release verification

For this completed study, the merge checks use the tests and `evaluation/verify_repository_release.py`. The default verifier checks repository files, curated endpoint tables, summary reconstruction, protocols, and protected source against the local base reference. It works without `runs/` and rejects tracked run artifacts. Optional `-archive-root` verification also reads the preserved raw checkpoints, logs, CSVs, and trajectories. Neither mode launches training, evaluation, calibration, or benchmarking.

Manual sources are maintained in separate Overleaf projects. The repository stores the three compiled PDFs; its release check verifies their presence, PDF headers, end markers, and cross-reference pointers without fixing their compiler-dependent bytes. Rendered content requires author review after compilation. Scientific evidence retains exact binary or newline-normalized text hashes.

13.4 Lightweight test allocation

Test area	Protected behavior
Environment	Gymnasium reset/step shape and dtype, seed repeatability, boundary clearance, forced success/collision, capacity versus active count, collision-penalty isolation, and finite random rollout.
Projection core	Obstacle-free identity/clipping, projection radius composition, active correction, and stationary failure action.
Wrapper topology	Physical projection diagnostics and proof that normalization is outside projection in the action path.
PPO rollout	Same-step autoreset, episode statistics, terminal projection diagnostics, and terminal solver-failure rejection.
Action-bound diagnostics	Counts, component rates, norm summaries, empty state, shape validation, and non-finite rejection.
Evaluation	Projection parameter propagation, slack averaging, action-bound metrics, failure-time unknown slack, and trajectory output.
Paired evaluation	Aligned off/on artifacts and pairing metadata.
Layout suites	Loading, padding, semantic hash independence from line endings, duplicate rejection, output refusal, provenance, and trajectory creation.
Training diagnostics	Cumulative/rolling outcomes, mutual exclusivity, complete projection exports, tag alignment, count consistency, and aggregation semantics.
Result aggregation	Checkpoint-before-seed hierarchy, missing coverage, duplicate keys, cross-seed key mismatch, non-finite evidence, nonempty output refusal, metadata discovery, and required training/trajectory evidence.
Orchestration	Thin launchers, exact training variant commands, weighted device selection, result-build order, and cleanup guards.

13.5 PASS-marker semantics

A PASS marker is written only after every required stage succeeds. The absence of a PASS marker is a failed or incomplete workflow, not an ambiguous state. A validation exception should be investigated and corrected; gates are not weakened to accept malformed evidence. Calibration additionally requires a manual scientific decision after automated PASS.

13.6 Verification responsibility

Verification status belongs to the generated summary, logs, audits, and manual-review record for the selected source revision and environment. A document build, partial test subset, or earlier PASS cannot replace that evidence.

13.7 Evidence separation

Regression and calibration fixtures preserve development checks and benchmark provenance. They are not experimental replicates. Contract changes require validation of the changed source version.

14 Traceability from mathematics to evidence

14.1 Contract traceability matrix

Mathematical or scientific contract	Primary implementation	Test or validator protection	Persisted evidence or downstream use
Fixed observation $6 + 5N_{\max}$ and inactive-slot zeroing	<code>environments/constrained_navigation.py</code>	Environment capacity and active-count tests; checkpoint compatibility and mismatch rejection	Checkpoint input dimension; trajectory states; layout evaluation provenance
Unicycle dynamics and angle wrapping	<code>ConstrainedNavigationEnv.step</code>	Reset/step, deterministic reset, finite rollout, forced outcomes	Episode CSV, positions/headings in NPZ, trajectory figures
Signed collision clearance	<code>_obstacle_margins</code> , <code>_get_info</code>	Boundary-clearance and forced-collision tests	Minimum clearance in CSV; state clearance in NPZ; safety tables
Reward and collision-penalty isolation	<code>ConstrainedNavigationEnv.step</code>	Collision-penalty difference test; short training validations	Returns in TensorBoard and CSV; high-penalty method metadata
Four-stage action map $a^{\text{sample}} \rightarrow a^{\text{bounded}} \rightarrow u^{\text{raw}} \rightarrow u^{\text{exec}}$	Actor, <code>NormalizedActionWrapper</code> , projection wrapper	Action-bound tests; wrapper order; trajectory physical-action reconstruction; pre-experiment artifact audit	Action-bound TensorBoard/CSV metrics; three action arrays in NPZ
Lookahead point and Jacobian	<code>compute_lookahead_point</code> , <code>compute_lookahead_jacobian</code>	Projection geometry tests and active correction scenarios	Projection parameter metadata; correction/slack trajectories
CBF rows $Au + \xi \geq b$	<code>build_cbf_constraints</code>	Projection-radius and active-constraint tests; trajectory active-count audit	Slot-aligned slack, active constraint count, solver status
Soft bounded projection QP	<code>solve_projection_qp</code>	Identity, clipping, active correction, failure fallback; runtime validator	Executed physical action, correction, slack, objective/status diagnostics
Stationary failure and unknown slack	Projection core/wrapper; evaluator; trajectory recorder	Failure tests, terminal training failure rejection, artifact audits	Failure count; NaN episode slack; NPZ active-slot NaN
PPO raw-sample likelihood	Agent, <code>ppo_continuous_action.py</code>	Rollout semantics tests; codebase validator; checkpoint smoke training	Old log probabilities in training; checkpoint; PPO loss events

Mathematical or scientific contract	Primary implementation	Test or validator protection	Persisted evidence or downstream use
Same-step autoreset and terminal data	Vector environment and <code>final_info</code> handling	Same-step autoreset/statistics tests; projection terminal diagnostics tests	Correct episode event counts and rollout burden totals
Action-bound clipping metrics	<code>ActionBoundDiagnostics</code> ; evaluator	Dedicated diagnostic tests; pre-experiment artifact audit	TensorBoard tags; episode CSV; training rollout table; figures
Projection training burden	<code>ProjectionTrainingDiagnostics</code>	Terminal inclusion, count matching, failure rejection	Projection TensorBoard tags; training rollout diagnostics; burden figures
Layout semantic identity	<code>layout_suite.py</code>	LF/CRLF hash test, duplicates, padding and provenance tests	Suite ID/SHA in CSV/NPZ and protocol audit
Same-checkpoint pairing	<code>evaluate_projection_pair.py</code>	Paired artifact test; historical regression audit	Paired episode and summary CSV; enabled-minus-disabled deltas
Trajectory $T + 1/T$ alignment	<code>trajectory_recording.py</code>	Recorder/evaluator tests and artifact validator	Auditable NPZ and trajectory figures
Checkpoint-first aggregation	<code>aggregate_projection_results.py</code>	Result aggregation tests for hierarchy and key consistency	Checkpoint summary, method summary, paired summaries
Training event conversion	<code>analysis/training_diagnostics.py</code>	Dedicated episode/rollout/tag-alignment tests	Raw scalar, episode safety, rollout burden, and curve-point CSVs
Saved-artifact-only plotting	<code>plot_projection_results.py</code> , build orchestrator	Required evidence tests; nonempty output refusal; figure and result-artifact completeness	Scientific diagnostic PDFs and CSV/JSON tables

14.2 Change-impact rules

A change to an observation field requires checkpoint compatibility review, environment tests, trajectory review, and protocol provenance review. A change to action mapping requires PPO likelihood review, clipping diagnostics, trajectory schema interpretation, projection tests, and all final checkpoints to be retrained. A change to projection failure semantics requires updates to wrapper info, episode metrics, NPZ validation, training fail-fast behavior, analysis rejection, and records. A change to aggregation requires new result tests and regeneration of every table and figure. This impact discipline is necessary because research validity crosses module boundaries even when the code change appears local.

15 Default parameters and rationale

Parameter	Value	Implemented role and rationale
<code>max_obstacles</code>	3	Fixes the observation capacity at 21 dimensions and matches the historical checkpoint. Three slots provide nontrivial geometry without making task generation the primary research problem.
<code>num_active_obstacles</code>	3	Activates all built-in training obstacles while retaining the ability to use fewer active slots in controlled layouts.
Δt	0.1	Provides a short, explicit discrete step for the compact unicycle simulation.
$v_{\max}$	1.0	Sets the physical forward-speed scale; physical units are arbitrary within the simulation.
$\omega_{\max}$	2.0	Allows visible turning in short episodes and yields nominal minimum turn radius $v_{\max}/\omega_{\max} = 0.5$ at maximum speed.
r_{goal}	0.25	Avoids requiring unrealistically precise final positioning.
r_{agent}	0.10	Gives the unicycle center a finite collision footprint.
$T_{\max}$	200	Corresponds to 20 simulated seconds and bounds episode cost.
c_p	1.0	Rewards reduction in Euclidean goal distance.
c_u	0.01	Penalizes physical speed and turn-rate effort quadratically.
c_{time}	0.01	Makes idling costly and encourages completion.
r_{goal}	10.0	Provides a distinct terminal success incentive.
$c_{\text{collision}}$	10.0	Default collision cost shared by baseline and projection training.
High collision penalty	50.0	Explicit reward-shaping comparison; no other reward term changes.
c_{timeout}	1.0	Penalizes remaining goal distance at time-limit truncation.
L	0.25	Places the barrier point ahead of the agent so turn rate influences local geometry.

Parameter	Value	Implemented role and rationale
α	2.0	Positive barrier-rate coefficient used in every active CBF row.
ρ	1000.0	Strongly discourages slack while preserving a softened feasible optimization problem.
r_{extra}	0.0	Default leaves projection geometry equal to collision geometry while preserving a separate projection-only parameter.
$\varepsilon_{\text{corr}}$	10^{-6}	Numerical threshold separating identity behavior from intervention.
Training launcher default budget	51,200 transitions	Default comparison budget exposed by the shared launchers; a frozen protocol may retain or override it explicitly.
Training seed	explicit nonnegative integer	Initializes one independent training run. The number of independently trained replicates is a protocol choice, not an obstacle count.
Evaluation episode limit	200	Default complete-episode limit shared with training and named-layout evaluation.
Layout repeats	1 by default	Positive repeat count supplied by the named-layout evaluator; a protocol freezes the campaign value.

The table contains implementation defaults and fixed model parameters. A campaign protocol must state which defaults it adopts or overrides. None of the values is presented as optimal, and any tuning after inspection of final outcomes requires an explicit development/final separation and a new frozen protocol.

16 Failure semantics and research incidents

16.1 Explicit failure table

Condition	Runtime behavior	Research consequence
Non-finite environment action	Raise before state update.	Run is invalid; no fallback transition is fabricated.
Non-finite observation or reward	Evaluator/trainer raises.	Indicates mechanics failure; artifact must not be accepted.
Projection solver failure during training	Stationary action is produced by projector, diagnostic accumulator raises immediately.	Training run is a technical incident and must not become a final checkpoint.
Projection solver failure during evaluation	Stationary action executes; failure count increments; active slack and episode slack summaries remain unknown.	Artifact may be retained for diagnosis but final protocol acceptance rejects solver failures.

Condition	Runtime behavior	Research consequence
No active obstacles	Projector returns clipped raw physical action and zero burden.	Valid identity path; clearance is undefined if no obstacle exists.
Successful zero slack	Numeric zeros retained in per-step and mean statistics.	Distinct from failure; demonstrates no relaxation was required.
Missing method-/seed/layout/mode	Result builder raises before aggregation.	No partial final table or figure is produced.
Mismatched checkpoint or suite hash	Artifact discovery/audit raises.	Evidence from different scientific identities cannot be combined.
Duplicate episode or layout key	Evaluator or analysis raises.	Prevents double counting.
Existing final output directory	Builder refuses to run.	Prevents stale generated files from surviving a new build.
Missing PASS marker	Workflow considered incomplete or failed.	Investigate logs; do not infer success from intermediate files.

16.2 Incident handling principle

A solver, schema, or identity failure is not converted into a favorable metric. Unknown quantities remain unknown, incomplete evidence blocks the final build, and historical fixtures are not silently substituted for missing final runs. This approach is intentionally stricter than ordinary exploratory scripting because silent artifact contamination is more damaging than a failed command.

17 Limitations and interpretation boundaries

- Training uses one deterministic three-obstacle geometry. The policy can specialize to that geometry.
- The core suite is fixed and small. It measures deterministic transfer and robustness, not broad distributional generalization.
- At most three obstacles are encoded in the present checkpoint-compatible observation.
- The workspace is unbounded; a poor policy may travel far from the intended route.
- Dynamics are a compact explicit-Euler unicycle simulation with no actuator lag, noise model, or hardware mismatch.
- The CBF condition is local, evaluated through a lookahead approximation, softened by slack, and applied in discrete time.
- The projection layer is not a route planner. It can prevent or delay collision without creating a globally successful trajectory.
- The Gaussian actor may rely on normalized action clipping before projection; clipping metrics must be interpreted separately from filter intervention.
- PPO uses raw sampled action likelihoods with rewards from projected transitions. The implementation is empirical and does not claim a corrected unbiased projected PPO derivation.
- Five independent training runs per history limit inference; the paper-level attribution interpretation is retrospective and descriptive.
- Regression and calibration checkpoints are fixtures, not experimental replicates.

- Repeated stochastic-policy evaluation on the fixed training geometry and deterministic named-layout transfer evaluation answer different questions and must be specified separately by protocol.

The repository can support later artifact-consuming work, but future modules should be additive consumers of exported states, actions, returns, values, and transitions. They should not retroactively redefine the present environment, projection, evaluation, or artifact contracts.

18 Change control and experiment freeze

18.1 Validity-threatening changes

Before a final experiment campaign, the selected source revision must pass consolidated validation and the final protocol set must be frozen. During final training and evaluation, code changes are allowed only for concrete validity-threatening defects. Every change must state:

- the defect and why it threatens validity;
- affected contracts, artifacts, tests, and records;
- whether existing checkpoints or evaluation evidence remain compatible;
- which methods, seeds, layouts, and modes must be rerun;
- the new source identity and validation evidence.

18.2 Contracts requiring explicit revision

At minimum, explicit review is required for changes to:

- observation ordering or obstacle capacity;
- Gaussian sampling, normalized clipping, or physical action mapping;
- wrapper nesting;
- dynamics, reward, success, collision, or timeout semantics;
- collision radius versus projection-only extra clearance;
- lookahead geometry, CBF rows, QP objective, bounds, or solver;
- stationary projection failure behavior or NaN semantics;
- PPO raw-action likelihood ownership or same-step autoreset handling;
- TensorBoard tag meanings or required training evidence;
- episode CSV fields, trajectory schema, or alignment;
- layout semantic identity, frozen suite contents, or pairing keys;
- checkpoint-first aggregation, paired-delta definitions, or seed statistics;
- saved-artifact-only result generation and required result-artifact completeness.

18.3 Freeze checklist

The final campaign should not begin until all of the following are true:

1. source compilation and the complete test suite pass in `RL_PROJECTS`;
2. consolidated pre-experiment validation writes its PASS marker;
3. validation figures receive focused manual inspection;
4. the device benchmark selects and records the final training device;
5. the core suite and its freeze record remain unchanged;
6. final training seeds, methods, budgets, projection modes, and evaluation protocols are machine-readable and frozen;
7. historical fixtures are separated from final checkpoint paths;
8. output directories are reserved and empty;
9. the repository revision and environment export are archived.

A Canonical command entry points

For completed-release verification with RL_PROJECTS active, run:

```
python -m pytest -q -rs
python -m evaluation.verify_repository_release --base-ref main --require-tracked
```

Use the second command after staging the intended files, including PDFs regenerated in Overleaf. The following entry points remain available for development and historical workflow reference; they are not additional merge requirements:

```
python -m pytest -q -rs
scripts\validate_pre_experiment_codebase.bat
scripts\inspect_pre_experiment_validation.bat
scripts\benchmark_training_devices.bat
scripts\train_ppo_baseline.bat [SEED] [TIMESTEPS] [CHECKPOINT] [--device ...]
scripts\train_ppo_high_penalty.bat [SEED] [TIMESTEPS] [CHECKPOINT] [--device ...]
scripts\train_ppo_projection.bat [SEED] [TIMESTEPS] [CHECKPOINT] [--device ...]
scripts\evaluate_policy.bat ...
scripts\evaluate_projection_pair.bat ...
scripts\calibrate_core_layouts.bat
scripts\plot_projection_results.bat PROTOCOL EVALUATION_DIR OUTPUT_DIR [RUNS_DIR]
```

Exact arguments remain defined by the Python modules and thin launchers. This appendix is an entry-point map, not a substitute for `-help` or the executable validation guide.

References

- [1] Aaron D. Ames et al. “Control Barrier Function Based Quadratic Programs for Safety Critical Systems”. In: *IEEE Transactions on Automatic Control* 62.8 (2017), pp. 3861–3876. DOI: 10.1109/TAC.2016.2638961.
- [2] Steven Diamond and Stephen Boyd. “CVXPY: A Python-Embedded Modeling Language for Convex Optimization”. In: *Journal of Machine Learning Research* 17.83 (2016), pp. 1–5. URL: <https://www.jmlr.org/papers/v17/15-408.html>.
- [3] Sébastien Gros, Mario Zanon, and Alberto Bemporad. “Safe Reinforcement Learning via Projection on a Safe Set: How to Achieve Optimality?” In: *IFAC-PapersOnLine* 53.2 (2020), pp. 8076–8081. DOI: 10.1016/j.ifacol.2020.12.2276.
- [4] Tuomas Haarnoja et al. “Soft Actor-Critic Algorithms and Applications”. In: *arXiv preprint arXiv:1812.05905* (2018). DOI: 10.48550/arXiv.1812.05905.
- [5] Shengyi Huang et al. “CleanRL: High-Quality Single-File Implementations of Deep Reinforcement Learning Algorithms”. In: *Journal of Machine Learning Research* 23.274 (2022), pp. 1–18. URL: <https://www.jmlr.org/papers/v23/21-1342.html>.
- [6] John Schulman et al. “High-Dimensional Continuous Control Using Generalized Advantage Estimation”. In: *International Conference on Learning Representations*. 2016. arXiv: 1506.02438.
- [7] John Schulman et al. “Proximal Policy Optimization Algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017). DOI: 10.48550/arXiv.1707.06347.
- [8] Bartolomeo Stellato et al. “OSQP: An Operator Splitting Solver for Quadratic Programs”. In: *Mathematical Programming Computation* 12.4 (2020), pp. 637–672. DOI: 10.1007/s12532-020-00179-2.